SALUS SECURITY

OCT 2026



CODE SECURITY ASSESSMENT

ASPECTA

Overview

Project Summary

- Name: Aspecta - Uniperp
- Platform: EVM-compatible chains
- Language: Solidity
- Repository:
  - https://github.com/aspecta-ai/contract-v1-core/tree/uniperp-beautify-patch/projects/uniperp
- Audit Range: See Appendix - 1

Project Dashboard

Application Summary

Name	Aspecta - Uniperp
Version	v2
Type	Solidity
Dates	Oct 16 2026
Logs	Oct 09 2026, Oct 16 2026

Vulnerability Summary

Total High-Severity issues	4
Total Medium-Severity issues	14
Total Low-Severity issues	4
Total informational issues	2
Total	24

Contact

E-mail: support@salusec.io

SALUS

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

SALUS

Content

Introduction **4**
1.1 About SALUS 4
1.2 Audit Breakdown 4
1.3 Disclaimer 4
Findings **5**
2.1 Summary of Findings 5
2.2 Notable Findings 7
1. Stale Funding Checkpoints Can Charge Pre-TGE Intervals After TGE 7
2. Short Settlement Bypasses Historical Funding Charges 8
3. Local Settlement Bypasses the Configured Chain 9
4. Settlement Rate Drift Misallocates Shared Inventory 10
5. Long Close Can Block Cross-Chain Short Settlement 11
6. Partial Open-Long Fills Create Unredeemable Deposits 12
7. Late Long Entries Dilute Terminal Funding 13
8. Native Fee Refunds Use tx.origin 14
9. Unsettled Shorts Can Lock Currency When No Long Exists 15
10. Reversible Long Locks Inflate Funding 16
11. Pre-End Claims Strand an Extra Fee Share 17
12. Tick History Can Overwrite Funding Snapshots 18
13. Late Cross-Chain Settlement Locks Submitted Tokens 19
14. Closing All Shorts Through Trading Blocks Long Settlement 20
15. Partial Short Settlement Discounts Remaining Keys 21
16. Cross-Chain Rate Changes Can Lock Short Settlement Tokens 22
17. Partial Settlement Payouts Burn Fully Valued Long Keys 23
18. Centralization risk 25
19. Fee Quote Offers an Unsupported Payment Mode 26
20. Blacklisted Recipients Cannot Remove Liquidity 27
21. Parameter Updates Rewrite Historical Funding 28
22. Retained ETH in the Batch Operator Can Be Spent by Subsequent Callers 29
2.3 Informational Findings 30
23. Router AccessControl Roles Are Unused 30
24. Administrative Changes Lack Events 31
Appendix **32**
Appendix 1 - Files in Scope 32

SALUS

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

SALUS

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Stale Funding Checkpoints Can Charge Pre-TGE Intervals After TGE	High	Business Logic	Resolved
2	Short Settlement Bypasses Historical Funding Charges	High	Business Logic	Resolved
3	Local Settlement Bypass the Configured Chain	High	Business Logic	Resolved
4	Settlement Rate Drift Misallocates Shared Inventory	High	Business Logic	Acknowledged
5	Long Close Can Block Cross-Chain Short Settlement	Medium	Business Logic	Acknowledged
6	Partial Open-Long Fills Create Unredeemable Deposits	Medium	Business Logic	Acknowledged
7	Late Long Entries Dilute Terminal Funding	Medium	Business Logic	Acknowledged
8	Native Fee Refunds Use tx.origin	Medium	Business Logic	Acknowledged
9	Unsettled Shorts Can Lock Currency When No Long Exists	Medium	Business Logic	Resolved
10	Reversible Long Locks Inflate Funding	Medium	Business Logic	Acknowledged
11	Pre-End Claims Strand an Extra Fee Share	Medium	Business Logic	Acknowledged
12	Tick History Can Overwrite Funding Snapshots	Medium	Business Logic	Acknowledged
13	Late Cross-Chain Settlement Locks Submitted Tokens	Medium	Business Logic	Acknowledged
14	Closing All Shorts Through Trading Blocks Long Settlement	Medium	Business Logic	Acknowledged
15	Partial Short Settlement Discounts Remaining Keys	Medium	Business Logic	Acknowledged
16	Cross-Chain Rate Changes Can Lock Short Settlement Tokens	Medium	Business Logic	Resolved
17	Partial Settlement Payouts Burn Fully Valued Long Keys	Medium	Business Logic	Mitigated
18	Centralization risk	Medium	Centralization	Acknowledged

SALUS

19	Fee Quote Offers an Unsupported Payment Mode	Low	Business Logic	Acknowledged
20	Blacklisted Recipients Cannot Remove Liquidity	Low	Business Logic	Acknowledged
21	Parameter Updates Rewrite Historical Funding	Low	Business Logic	Acknowledged
22	Retained ETH in the Batch Operator Can Be Spent by Subsequent Callers	Low	Business Logic	Acknowledged
23	Router AccessControl Roles Are Unused	Informational	Redundancy	Acknowledged
24	Administrative Changes Lack Events	Informational	Logging	Acknowledged

SALUS

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Stale Funding Checkpoints Can Charge Pre-TGE Intervals After TGE

Severity: High	Category: Business Logic
Target: - projects/uniperp/contracts/uniperpPool/UniperpPool.sol	

Description

The first synchronization after TGE counts every elapsed rate interval since `_lastRateUpdateTime`, but the TGE adjustment removes at most one pre-TGE interval. For example, if the last update was at 1,000, TGE is at 2,000, the interval is 100 seconds, and synchronization occurs at 2,101, the function charges ten intervals even though only one interval ended after TGE. The compounded charge reduces existing short collateral and credits existing longs for periods when funding should not have accrued; a long holder can trigger the update through claimFundingFee.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L54-L62

```
function _calculateFundingFeeRateBps(
    uint256 lastRateUpdateTime,
    uint256 intervals
) internal view returns (uint256 rateBps) {
    if (tgeTime == 0 || block.timestamp <= tgeTime) {
        return 0;
    }

    if (
        lastRateUpdateTime < tgeTime &&
        (tgeTime - _lastRateUpdateTime) % rateUpdateInterval == 0
    ) {
        intervals = Math.safeSub(intervals, 1);
    }

    if (intervals == 0) {
        return 0;
    }
    ...
}
```

Recommendation

Calculate the funding-specific interval count from `max(lastRateUpdateTime, tgeTime)` instead of subtracting a single interval from the general update count.

Status

This issue has been resolved by the team with commit 4d08e5a.

SALUS

2. Short Settlement Bypasses Historical Funding Charges

Severity: High	Category: Business Logic

Target:
- projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

The Pool calculates funding according to the shortage between required settlement-token reserves and Router inventory. Funding is updated lazily: _updateRates() counts the complete intervals elapsed since the previous checkpoint, reads the current Router balance, and compounds the resulting shortage rate for all those intervals. It does not retrieve an inventory snapshot for each historical interval.

During short settlement, the Router first executes st.balance += tokenAmount and only then calls Pool.settleShort(). The Pool begins that callback with _sync(user). Its historical funding catch-up consequently observes inventory after the settlement delivery, even though that inventory was unavailable during the elapsed intervals.

For example, assume required reserves remain 100 while Router inventory remains zero for three eligible funding intervals, with no intervening synchronization. A short holder then delivers 100 tokens through the public settlement path. The Router records a balance of 100 before invoking the Pool, which calculates a zero shortage rate and applies it to all three intervals. The short holder retains collateral that should have funded the longs during the shortage period. A partial replenishment similarly reduces the historical charge.

Recommendation

Before the Router credits new claimable inventory, have it call a restricted Pool checkpoint that accounts for elapsed funding using the old Router balance. Then credit the new inventory and perform the short settlement callback within the same transaction.

Status

This issue has been resolved by the team with commit 4d08e5a.

SALUS

3. Local Settlement Bypasses the Configured Chain

Severity: High	Category: Business Logic

Target:
- projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol

Description

The local Router settlement entry point resolves only a token address to a Pool and does not enforce that the configured settlement endpoint is the local chain. A same-address local token or OFT can therefore be accepted even when the configuration intends settlement to originate on a remote chain.

The Pool closes the short using locally credited inventory, while long claims may be paid from a remote Dispatcher. This breaks the intended inventory-location invariant and can cause remote-payment failure or raw-unit accounting errors where deployments use incompatible token representations.

projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol:L79-L87

```
function settleShort(address token, uint256 amount) external {
    if (amount == 0) revert ZeroAmount();
    address pool = settlements[token].pool;
    if (pool == address(0)) revert PoolNotFound();
    // transfer token into this contract.
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    _settleShort(msg.sender, pool, token, amount);
}
```

Recommendation

Reject local settlement when the configured endpoint is remote, or perform an explicit OFT bridge/conversion before crediting the Pool inventory. Normalize token decimals at the settlement boundary and bind each accepted inventory credit to its configured source chain.

Status

This issue has been resolved by the team with commit 4d08e5a.

SALUS

4. Settlement Rate Drift Misallocates Shared Inventory

Severity: High	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

Long settlement records the current adjusted token-per-key rate in the user's locked position, while short settlement uses the rate at the time it executes. These rates can differ as `totalShortDeposits` changes, but both sides rely on the same Router Token inventory.

If shorts deliver Token at a higher rate and longs later claim at a lower rate, surplus Token can remain after all long positions are cleared. In the reverse order, longs who claim early can receive Token at their older, higher rate, leaving less for later claimants.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L1209-L1244

```
function settleLong(uint256 keyAmount) external {
    _positionData.closeLong(msg.sender, keyAmount, depositAmount);
    // increase the pending claim tokens.
    _positionData.updateLockedPosition(
        msg.sender,
        PositionManager.SettleCustody({
            keyAmount: locked.custody.keyAmount + keyAmount,
            longDeposit: locked.custody.longDeposit + depositAmount,
            unlockTime: block.timestamp + settleLockPeriod,
            tokenPerLongKey: (settleParams.tokenPerLongKey * _settleAdjRateBps) / 1e4
        }),
        0
    );
}
```

Recommendation

Use a consistent settlement rate for both sides, or reserve the exact Token amount owed when each long position is locked. Rate changes must also reconcile existing locked claims with available Router inventory.

Status

This issue has been acknowledged by the team.

SALUS

5. Long Close Can Block Cross-Chain Short Settlement

Severity: Medium	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

A short holder pays `C` USDT when opening a position, but the Pool records a logical `shortDeposit` of `2C`. Settling the short requires the Pool to refund that deposit from its shared USDT balance. In a Pool with one long and one short, the long's funds provide the additional refund capacity; no USDT is reserved while a cross-chain settlement message is in transit.

The short holder may submit TGE Tokens on the source chain while the long position is still open. Before the message reaches BSC, the long holder can close through `trade` and withdraw USDT. The destination `settleShort` then fails when it tries to pay the short's refund, leaving the short open while the user's TGE Tokens remain held by the source-chain Dispatcher.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L1165-L1198

```
function settleShort(address user, uint256 tokenAmount) external {
    // Full close takes remaining deposit to avoid floor dust from div-then-mul.
    ValueTracker.ShortDeposit memory shortDeposit =
shortPos.shortDeposit.mul(keyAmount).div(
        shortPos.keyAmount
    );
    uint256 shortDepositAmount = shortDeposit.getAmountAt(
        _stateData.fee.accShortFundingFeeRate
    );

    _positionData.closeShort(user, keyAmount, shortDeposit);
    _stateData.totalShortDeposits = _stateData.totalShortDeposits.sub(shortDeposit);

    uint256 protocolFee = _protocolFee(shortDepositAmount, PROTOCOL_FEE_BPS_UNI);
    uint256 refund = _processTransfer(user, shortDepositAmount, 0, protocolFee);
}
```

Recommendation

The Pool should reserve the USDT needed for an in-flight short settlement so that intervening trades cannot withdraw it. If settlement still cannot complete, the Dispatcher should provide a way to return the submitted TGE Tokens.

Status

This issue has been acknowledged by the team.

SALUS

6. Partial Open-Long Fills Create Unredeemable Deposits

Severity: Medium	Category: Business Logic
Target: - projects/uniperp/contracts/uniperpPool/UniperpPool.sol	

Description

When an open-long liquidity order is partially filled, StateManager.removeUniLiquidity() calculates the consumed currency and acquired keys separately. The acquired key amount is rounded down to an integer, while the corresponding costAmount remains payable. Consequently, a fill worth less than one whole key can produce a positive costAmount and a zero keyAmount.

UniperpPool._executeRemoveUniLiquidity() then deletes the liquidity order and calls PositionManager.openLong(recipient, 0, costAmount). The user receives the refund for the unfilled portion, but the consumed currency becomes a static long deposit without any associated key.

The user can therefore lose the redeemable value of a fractional fill when removing the order.

Recommendation

Preserve fractional fill value as a redeemable entitlement instead of creating a long deposit with zero keys.

Status

This issue has been acknowledged by the team.

SALUS

7. Late Long Entries Dilute Terminal Funding

Severity: Medium	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

Trading remains open after TGE until `tradeEndTime`. When trading ends, the Pool distributes the remaining effective short deposits across the long deposits that exist at that moment, without accounting for how long each position was open.

A user can open a long shortly before the deadline and receive a share of the terminal funding, even though earlier long holders bore the settlement-token shortage for longer. This dilutes the funding received by those earlier holders.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L227-L252

```
function _setEndState() internal {
    ...
    uint256 totalShortDeposits = _stateData.totalShortDeposits.getAmountAt(
        _stateData.fee.accShortFundingFeeRate
    );
    int256 biC = _stateData.biLiquidityCurrency;
    // update total short deposit.
    totalShortDeposits = biC >= 0
        ? totalShortDeposits + uint256(biC)
        : Math.safeSub(totalShortDeposits, uint256(-biC));
    // FEE_RATE_DENOMINATOR = 1e4
    _stateData.fee = _stateData.fee.update(
        ValueTracker.FEE_RATE_DENOMINATOR,
        _stateData.totalLongDeposits,
        totalShortDeposits
    );
    settleParams.tokenPerLongKey = _oiKeys() == 0
        ? 0
        : IUniperpLZRouter(lzRouter).getSettledTokenBalance() / _oiKeys();
    settleLockPeriod = 0;
}
```

Recommendation

Distribute funding using checkpoints that reflect when each long position was open. Alternatively, stop new long entries before the terminal funding calculation.

Status

This issue has been acknowledged by the team.

SALUS

8. Native Fee Refunds Use `tx.origin`

Severity: Medium	Category: Business Logic
Target: - projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol	

Description

For remote long claims, the Router passes `tx.origin` to LayerZero as the recipient of any unused native messaging fee. In a smart-account or relayer flow, `tx.origin` may be a bundler or relayer rather than the user who funded the claim.

Any unused fee can therefore be refunded to that third party. The TGE Token claim is unaffected, but the claimant loses the refundable portion of the native fee.

projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol:L79-L87

```
function transferTokens(
    address to,
    uint256 amount,
    uint256 protocolFee,
    bytes calldata options
) external payable {
    _lzSend(
        s.eid,
        message,
        combineOptions(s.eid, SEND, options),
        MessagingFee(msg.value, 0),
        payable(tx.origin)
    );
}
```

Recommendation

Pass the claimant's address explicitly from the Pool and use it as the LayerZero refund recipient.

Status

This issue has been acknowledged by the team.

SALUS

9. Unsettled Shorts Can Lock Currency When No Long Exists

Severity: Medium	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

If short holders do not settle before `tradeEndTime`, they cannot settle afterward because `UniperpPool.settleShort()` requires the trading period to remain open. Ordinary trading also closes at that time.

When there are no long positions, the terminal funding update has no long deposits to credit and returns without distributing the remaining short-side balance. With no post-end short exit, the associated USDT remains in the Pool without a normal withdrawal path.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L1165-L1198

```
function settleShort(address user, uint256 tokenAmount) external {
    // Only lz router can settle short.
    _checkCaller(lzRouter);
    _checkTradePeriod();
    ...
}
```

Recommendation

Provide a post-end finalization path for unsettled shorts when no long claimant exists. The protocol should define who can withdraw the remaining USDT and under what conditions.

Status

This issue has been resolved by the team with commit 3283cef.

SALUS

10. Reversible Long Locks Inflate Funding

Severity: Medium	Category: Business Logic

Target:
- projects/uniperp/contracts/libraries/PositionManager.sol

Description

The function `UniperpPool.settleLong()` adds the locked position's Token claim to `totalPendingClaimTokens`. The funding calculation treats this pending claim as required settlement-token reserves, so locking keys can increase the measured inventory shortage.

A long holder can lock keys, allow a funding interval to charge shorts at the higher shortage rate, and then complete `cancelSettleLong()`. Cancellation removes the pending claim, but does not reverse the funding already credited to longs, including the locked holder.

projects/uniperp/contracts/libraries/PositionManager.sol:L51-L65

```
function updateLockedPosition(
    PositionData storage self,
    address account,
    SettleCustody memory custody,
    uint256 settleCancelCooldownUntil
) internal {
    self.totalPendingClaimTokens =
        self.totalPendingClaimTokens +
        (custody.keyAmount * custody.tokenPerLongKey) - // New pending claim tokens
        (self.lockedPositions[account].custody.keyAmount *
            self.lockedPositions[account].custody.tokenPerLongKey); // Old pending claim
tokens

    self.lockedPositions[account].custody = custody;
    self.lockedPositions[account].settleCancelCooldownUntil = settleCancelCooldownUntil;
}
```

Recommendation

Count only irrevocable Token claims as reserve demand, or reverse funding attributable to a claim when its lock is cancelled.

Status

This issue has been acknowledged by the team.

SALUS

11. Pre-End Claims Strand an Extra Fee Share

Severity: Medium	Category: Business Logic
Target: - projects/uniperp/contracts/uniperpPool/UniperpPool.sol	

Description

Before trading ends, `UniperpPool.claimSettledLong()` reduces the user's Token payment by twice the protocol fee, but instructs `UniperpLZRouter.transferTokens()` to pay the protocol only once. The Router therefore deducts `claimTokenAmount - protocolFee` from its inventory, leaving one fee share unallocated.

If all longs claim before trade end, no long OI remains to receive that surplus through the terminal settlement rate. The extra TGE Tokens remain in Router inventory, unpaid to either users or the protocol.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L1281-L1340

```
function claimSettledLong(
    bytes calldata options
) external payable returns (uint256 claimTokenAmount, uint256 keysPaid) {
    IUniperpLZRouter(lzRouter).transferTokens{value: msg.value}(
        msg.sender,
        claimTokenAmount - (isTradeEnded ? 1 : 2) * protocolFee,
        protocolFee,
        options
    );
}
```

Recommendation

Deduct one protocol fee from the user's payment, or explicitly transfer the second fee to its intended recipient. The Pool and Router should account for the full `claimTokenAmount` on every claim.

Status

This issue has been acknowledged by the team.

SALUS

12. Tick History Can Overwrite Funding Snapshots

Severity: Medium	Category: Business Logic
Target: - projects/uniperp/contracts/libraries/Ticks.sol	

Description

The function `Ticks.pushHistory()` stores fee snapshots in a circular buffer limited to 1,000 entries per tick. A liquidity position can remain open after the snapshots needed to calculate its funding have been overwritten.

If a boundary tick records more than 1,000 distinct fee snapshots before the position is removed, `Ticks.getNextHistory()` can no longer retrieve the earliest crossing after that position was added. The removal calculation may then use a later snapshot and pay too much or too little funding.

projects/uniperp/contracts/libraries/Ticks.sol:L115-L148

```
function pushHistory(
    mapping(uint32 tickId => TickData) storage self,
    uint32 tickId,
    uint256 timestamp,
    ValueTracker.Fee memory fee
) internal {
    uint16 last = (index + HISTORY_CAPACITY - 1) % HISTORY_CAPACITY;
    // add one new element.
    self[tickId].history.timestamp[index] = timestamp;
    self[tickId].history.fee[index] = fee;
    // update the index.
    self[tickId].history.index = (index + 1) % HISTORY_CAPACITY;
    self[tickId].historyNeeded = 0;
}
```

Recommendation

Preserve the history needed by live positions, or detect when a position's required snapshots have expired and stop removal from silently using incomplete history.

Status

This issue has been acknowledged by the team.

SALUS

13. Late Cross-Chain Settlement Locks Submitted Tokens

Severity: Medium	Category: Business Logic

Target:
- projects/omni-hub/contracts/UniperpLZRouter/UniperpTokenDispatcher.sol

Description

The function `UniperpTokenDispatcher.settleShort()` takes the user's TGE Tokens and sends an asynchronous message to BSC. The destination `UniperpPool.settleShort()` accepts settlement only during the trading period, but the source call does not ensure that the message will arrive before `tradeEndTime`.

If the message arrives after trading ends, the Pool reverts and every retry fails the same time check. The source Dispatcher still holds the user's TGE Tokens, while the short position remains unsettled; it provides no user cancellation or withdrawal path.

projects/omni-hub/contracts/UniperpLZRouter/UniperpTokenDispatcher.sol:L57-L74

```
function settleShort(address token, uint256 amount, bytes calldata options) external
payable {
    if (amount == 0) revert ZeroAmount();
    if (token == address(0)) revert ZeroAmount();

    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);

    _settledTokenAmount[token][msg.sender] += amount;

    bytes memory message = _encodeSettleMessage(msg.sender, token, amount);
    _lzSend(
        dstEid,
        message,
        combineOptions(dstEid, SEND, options),
        MessagingFee(msg.value, 0),
        payable(msg.sender) // refund native token.
    );
}
```

Recommendation

Use a consistent settlement rate for both sides, or reserve the exact Token amount owed when each long position is locked. Rate changes must also reconcile existing locked claims with available Router inventory.

Status

This issue has been acknowledged by the team.

SALUS

14. Closing All Shorts Through Trading Blocks Long Settlement

Severity: Medium	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

Short holders can close their positions through ordinary `trade()` without delivering TGE Tokens to the Router. If all shorts close this way, Router inventory can remain zero while long positions are still open.

At trade end, `UniperpPool._setEndState()` sets `tokenPerLongKey` to zero because it divides the Router balance by the remaining long keys. Long holders can no longer trade-close, and `settleLong()` rejects the zero rate, leaving their keys without a settlement path.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L237-L246

```
function _setEndState() internal {
    ...
    totalShortDeposits = biC >= 0
        ? totalShortDeposits + uint256(biC)
        : Math.safeSub(totalShortDeposits, uint256(-biC));
    // FEE_RATE_DENOMINATOR = 1e4
    // convert all total short deposits into the long side fees.
    _stateData.fee = _stateData.fee.update(
        ValueTracker.FEE_RATE_DENOMINATOR,
        _stateData.totalLongDeposits,
        totalShortDeposits
    );

    settleParams.tokenPerLongKey = _oiKeys() == 0
        ? 0
        : IUniperpLZRouter(lzRouter).getSettledTokenBalance() / _oiKeys();
    settleLockPeriod = 0;
}
```

Recommendation

Provide a long-side exit when settlement inventory is zero, or require the Token reserve to be funded before trading closes. Do not make the only post-end redemption path depend on a nonzero Router balance.

Status

This issue has been acknowledged by the team.

SALUS

15. Partial Short Settlement Discounts Remaining Keys

Severity: Medium	Category: Business Logic

Target:
- projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol:L79-L87

Description

The function `UniperpPool.settleShort()` reduces `totalShortDeposits`, while the long-side key amount used by `_settleAdjBase()` remains unchanged. After the next rate-update interval, the lower short-deposit ratio can reduce `_settleAdjRateBps`.

A large short holder can settle part of a position first, wait for the rate to fall, and then settle the remaining keys using fewer TGE Tokens per key. The discount is limited by the configured adjustment floor, but grows with the size of the remaining position.

projects/uniperp/contracts/UniperpPool/UniperpPool.sol:L119-L125

```
function _settleAdjBase() internal view returns (uint256 adjBase) {
    if (_oiKeys() == 0 || _stateData.emaPrice == 0) return 200;
    uint256 feePerOpenInterest = _stateData.totalShortDeposits.getAmountAt(
        _stateData.fee.accShortFundingFeeRate
    ) / _oiKeys();
    adjBase = (100 * feePerOpenInterest) / _stateData.emaPrice;
}
```

Recommendation

Prevent a holder’s partial settlement from lowering the rate applied to its own remaining keys. For example, lock the settlement rate for a short position when its first settlement begins.

Status

This issue has been acknowledged by the team.

SALUS

16. Cross-Chain Rate Changes Can Lock Short Settlement Tokens

Severity: Medium	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

A user may submit the exact number of TGE Tokens needed to settle all remaining short keys at the current adjustment rate. The cross-chain message carries only `tokenAmount`; `UniperpPool.settleShort()` calculates the key amount using the rate when the message arrives.

If the rate falls during transit, the same Token amount converts to more keys than the user holds, and `InsufficientSettleKeys` causes the destination call to revert. The source Dispatcher has already taken the Tokens and offers no cancellation or withdrawal path.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L1165-L1198

```
function settleShort(address user, uint256 tokenAmount) external {
    // Only lz router can settle short.
    _checkCaller(lzRouter);
    _checkTradePeriod();
    _sync(user);
    if (tokenAmount == 0) revert ZeroAmount();
    if (settleParams.tokenPerShortKey == 0) revert InvalidSettlementConfig();
    // Here we use the round down. This is correct.
    uint256 keyAmount = tokenAmount /
        ((settleParams.tokenPerShortKey * _settleAdjRateBps) / 1e4);

    PositionManager.ShortPosition memory shortPos =
_positionData.staticShortPositions[user];
    if (shortPos.keyAmount == 0) revert NotShortPosition();
    if (keyAmount == 0 || keyAmount > shortPos.keyAmount) revert
InsufficientSettleKeys();
    ...
}
```

Recommendation

Include the intended key amount and an acceptable rate bound in the cross-chain message. If the destination cannot honor them, provide a way to return the Tokens held by the Dispatcher.

Status

This issue has been resolved by the team with commit 4d08e5a.

SALUS

17. Partial Settlement Payouts Burn Fully Valued Long Keys

Severity: Medium	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

In UniperpPool.claimSettledLong(), claimTokenAmount is capped at the Router's available tokeninventory. The function then calculates keysPaid = roundUpDiv(claimTokenAmount, rate). Rounding a partial token payment upward treats it as payment for a whole key, and the function removes that key and its associated long deposit.

If the settlement rate is 100 tokens per key and the Router holds 101 tokens, a claimant with two keys receives 101 tokens but loses both keys. The second key has received only one of its 100 owed tokens. An inventory of 99 tokens likewise consumes one whole key. Earlier claimants can leave shared inventory at such a non-integral key boundary, so later claimants can permanently lose an unpaid token entitlement.

This is not merely a delay in payout: claimSettledLong() removes keysPaid from the custody and global position accounting without recording the unpaid portion.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L1060-L1067

```
function claimSettledLong(
    bytes calldata options
) external payable returns (uint256 claimTokenAmount, uint256 keysPaid) {
    ...
    uint256 rate = isTradeEnded ? settleParams.tokenPerLongKey :
custody.tokenPerLongKey;
    uint256 tokenNeeded = custody.keyAmount * rate;
    uint256 available = IUniperpLZRouter(lzRouter).getSettledTokenBalance();
    claimTokenAmount = Math.min(tokenNeeded, available);
    keysPaid = rate == 0 ? 0 : Math.roundUpDiv(claimTokenAmount, rate);
    uint256 depositPaid = (custody.longDeposit * keysPaid) / custody.keyAmount;
    uint256 remainingKeyAmount = Math.safeSub(custody.keyAmount, keysPaid);
    _stateData.totalLongDeposits = Math.safeSub(_stateData.totalLongDeposits,
depositPaid);
    _stateData.openInterest = Math.safeSub(
        _stateData.openInterest,
        keysPaid * StateManager.OI_WAD
    );
}
```

Recommendation

Only settle keys that the available token balance can fully cover. Calculate keysPaid =

SALUS

min(custody.keyAmount, available / rate) and pay exactly keysPaid * rate. Preserve all unpaid keys and their associated deposits for later claims.

Status

This issue has been mitigated by the team. The team stated that key amounts will use two decimal places of precision to mitigate the impact of this issue.

SALUS

18. Centralization risk

Severity: Medium	Category: Centralization

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

In the UniperpPool/UniperpPoolFactory etc contracts, there exists some privileged roles, for example `protocolFeeDestination`. These roles have the authority to execute some key functions such as `setAdmin`, `setSettleParams`, etc.

If these roles' private keys are compromised, an attacker could trigger these functions to harm the whole protocol.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L922-L925

```
function setAdmin(address newAdmin) external {
    _checkCaller(protocolFeeDestination);
    admin = newAdmin;
}
```

Recommendation

We recommend transferring ownership to a multi-signature account protected by a timelock. This ensures that no single party can unilaterally redirect treasury flows and gives users advance notice of treasury changes.

Status

This issue has been acknowledged by the team.

SALUS

19. Fee Quote Offers an Unsupported Payment Mode

Severity: Low	Category: Business Logic
Target: - projects/omni-hub/contracts/UniperpLZRouter/UniperpTokenDispatcher.sol	

Description

The function `UniperpTokenDispatcher.quoteSettleShort()` accepts `payInLzToken` and can return a LayerZero Token fee quote. The corresponding `settleShort()` call always pays the messaging fee with native currency and sets the LZ Token fee to zero.

A frontend using `payInLzToken = true` may show a quote that the settlement transaction cannot use, causing an incorrect payment request or a failed transaction.

projects/omni-hub/contracts/UniperpLZRouter/UniperpTokenDispatcher.sol:L86-L95

```
function quoteSettleShort(
    address token,
    uint256 amount,
    bytes calldata options,
    bool payInLzToken
) public view returns (MessagingFee memory fee) {
    bytes memory message = _encodeSettleMessage(msg.sender, token, amount);
    fee = _quote(dstEid, message, combineOptions(dstEid, SEND, options), payInLzToken);
}
```

Recommendation

Remove the unsupported quote option, or implement LZ Token payment in `settleShort()` so its execution matches the quoted mode.

Status

This issue has been acknowledged by the team.

SALUS

20. Blacklisted Recipients Cannot Remove Liquidity

Severity: Low	Category: Business Logic
Target: - projects/uniperp/contracts/uniperpPool/UniperpPool.sol	

Description

The function `UniperpPool.removeUniLiquidity()` pays the resulting trade currency to the position's recipient through `_processTransfer()`. If that currency is USDT and the recipient is blacklisted, the token rejects the transfer.

The transfer failure reverts the entire removal transaction. The user cannot withdraw the position's USDT proceeds while the address remains blacklisted.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L667-L791

```
function _executeRemoveUniLiquidity(
    address recipient,
    bytes32 uniLiquidityPositionId
)
{
    ...
    refundAmount = _processTransfer(recipient, removeResult.costAmount, 0, protocolFee);
    ...
}
```

Recommendation

Allow a position owner to designate a different payout address for removal, subject to appropriate authorization checks.

Status

This issue has been acknowledged by the team.

SALUS

21. Parameter Updates Rewrite Historical Funding

Severity: Low	Category: Business Logic

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

The function `UniperpPool.setSettleParams()` replaces funding-related parameters immediately, without recording when the new values take effect. The next `_updateRates()` uses those values to calculate every complete interval since the previous checkpoint.

If several hours passed without a sync, one parameter change can alter funding for all those past intervals. Even calling `_sync()` in the setter would not fully solve a change made halfway through an interval, because funding is calculated only for complete intervals.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L921-L925

```
function setSettleParams(SettleParams calldata params) external {
    _checkCaller(protocolFeeDestination);
    settleParams = params;
}
```

Recommendation

Apply new parameters from the next interval boundary, or split the interval at the change time so each period uses the parameters that were active then.

Status

This issue has been acknowledged by the team.

SALUS

22. Retained ETH in the Batch Operator Can Be Spent by Subsequent Callers

Severity: Low	Category: Business Logic
Target: - projects/uniperp/contracts/uniperpBatch/BatchTxOperator.sol	

Description

BatchTxOperator.execute() forwards the specified ops[i].value with each operation but does not restrict the total forwarded ETH to the current caller's msg.value. Any ETH already held by the contract can therefore fund a subsequent caller's batch.

The recipient check only requires the operation's first calldata argument to equal the current caller. It does not establish ownership of the ETH being spent. Since operation targets are unrestricted, a later caller can submit a compatible call to an address they control and forward ETH retained from an earlier transaction.

Recommendation

Limit each batch's total forwarded ETH to its own msg.value and refund the unused portion of that contribution to the caller.

Status

This issue has been acknowledged by the team.

SALUS

2.3 Informational Findings

23. Router AccessControl Roles Are Unused

Severity: Informational	Category: Redundancy

Target:
- projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol

Description

The function `UniperpLZRouter.initialize()` grants `DEFAULT_ADMIN_ROLE` and `UNIPERP_ADMIN_ROLE`, but the Router does not use either role to authorize its administrative functions. Those functions rely on `onlyOwner` or OApp controls.

The grants add storage and gas costs and may mislead operators about which accounts can administer the Router. Holding either role does not provide the authority its name suggests.

projects/omni-hub/contracts/UniperpLZRouter/UniperpLZRouter.sol:L79-L87

```
function initialize(address _defaultAdmin, address _initialOwner) public initializer {
    __Ownable_init(_initialOwner);
    __OApp_init(_defaultAdmin);
    __UUPSUpgradeable_init();
    __AccessControl_init();
    _grantRole(DEFAULT_ADMIN_ROLE, _initialOwner);
    _grantRole(UNIPERP_ADMIN_ROLE, _defaultAdmin);
}
```

Recommendation

Remove the unused AccessControl roles, or use them consistently for the functions they are intended to govern.

Status

This issue has been acknowledged by the team.

SALUS

24. Administrative Changes Lack Events

Severity: Informational	Category: Logging

Target:
- projects/uniperp/contracts/uniperpPool/UniperpPool.sol

Description

Several privileged Pool functions change configuration without emitting events, including `setPoolConfig()`, `setPriceLimit()`, `setSettleParams()`, `modifyBiLiquidity()`, and `setAdmin()`.

Event-based monitors cannot promptly detect changes to the trading window, settlement rules, price limits, base liquidity, or admin address. Users must inspect transactions or historical storage to reconstruct these changes.

projects/uniperp/contracts/uniperpPool/UniperpPool.sol:L879-L896

```
function setPoolConfig(
    uint256 _tradeStartTime,
    uint256 _tradeEndTime,
    uint256 _rateUpdateInterval,
    uint256 _protocolFeePercentage,
    uint256 _tgeTime,
    uint256 _settleLockPeriod,
    uint256 _settleCancelCooldown
   ) external {
    _checkCaller(admin);
    tradeStartTime = _tradeStartTime;
    tradeEndTime = _tradeEndTime;
    rateUpdateInterval = _rateUpdateInterval;
    protocolFeePercentage = _protocolFeePercentage;
    tgeTime = _tgeTime;
    settleLockPeriod = _settleLockPeriod;
    settleCancelCooldown = _settleCancelCooldown;
}
```

Recommendation

Emit a dedicated event for each administrative update, including the new values and, where useful, the previous values.

Status

This issue has been acknowledged by the team.

SALUS

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit f19692d:

File	SHA-1 hash
UniperpPoolFactory.sol	64d049a67c25ad7d4f5c6c7f22e98073a976a035
UniperpPoolFactoryStorage.sol	da72ba15a1898daf1ac078c95522987d0c3b800b
IUniperpPoolFactory.sol	81210a863b1898aafba322b7776bdfbd77007463
UniperpPoolStorage.sol	91bac532868df6098f51514dcc79ebe1d9fadbaa
UniperpPool.sol	950bef4c4d788130cbd2dcfbdf8f6a3733f80c33
StateManager.sol	7be2d096cf5f3a38388ba9c79c17b0771e3d1824
Ticks.sol	cd9f9bde8a756d89e97748c307bb80179f4ca4df
Math.sol	bf9b69cfa7370003ca3994adc161523e84a0e8a9
ValueTracker.sol	5afc4eaa9bb9057ac79ca43ee0a9e9446954eda4
AddressUtils.sol	4214f45301199da2af6527fc9a436df279ea6a21
PositionManager.sol	242cef361129ec1003aa4492f28b3651f1ae6066
BatchTxOperator.sol	8ec1a5c274b8496d9c5661d8bf7f21e51b071da7
UniperpLZRouter.sol	82a35f42ff6a748995c61832492f8d371bd5ee79
UniperpLZRouterStorage.sol	f1bd21cfa3955fa0cbddff99e8b80497e02c1186
UniperpTokenDispatcherStorage.sol	3e072657ab48d2a69e57ffd227c1476f91073f8b
UniperpTokenDispatcher.sol	5e19c3141b1b43d9ac42991033ffeefe0f5f81ff

SALUS